Relatório Técnico: Implementação de Pipeline RISC-V

Arthur Carvalho Rodrigues, Débora Luiza de Paula Silva,
Gustavo Henrique Rodrigues de Castro e Mariana Almeida Mendonça

29 de abril de 2026

1 Introdução

Este relatório detalha a implementação de melhorias no processador RISC-V com pipeline de 5 estágios: Busca (IF), Decodificação (ID), Execução (EX), Memória (MEM) e Escrita (WB). O projeto focou na resolução de *Hazards* de controle e dados, visando a integridade da execução matemática e a otimização do desempenho geral do sistema, medido através do índice de Ciclos por Instrução (CPI).

2 Parte 1: Hazard de Controle (BEQ)

2.1 Análise do Problema

Na execução original, o processador falhava ao executar o desvio condicional (*branch*). O erro ocorria porque a Unidade de Branch (`BranchUnit`) não calculava o endereço de destino considerando o *Program Counter* (PC) atual. Consequentemente, o pipeline não realizava o *flush* (limpeza) das instruções carregadas indevidamente. O log de depuração abaixo demonstra que instruções subsequentes que deveriam ser descartadas foram executadas incorretamente, alterando o estado da máquina:

```
FAIL: x5 expected 0 because branch should flush it, got 99
Cycles : 14 | Instructions: 9 | CPI: 1.56
```

2.2 Implementação e Validação

A solução foi implementada modificando a lógica combinacional no arquivo `BranchUnit.v`. O cálculo do endereço alvo (`branch_target`) foi corrigido para realizar a soma do PC no estágio de execução (`pc_ex`) com o valor imediato extraído da instrução (`branch_imm`). Adicionalmente, o sinal `branch_taken` foi ativado quando a condição de igualdade dos registradores foi satisfeita.

```
    always @(*) begin
        branch_taken  = 1'b0;
        branch_target = pc_ex + 32'd4;
        //parte implementa  o beq
        if(opcode == BEQ) begin
            if(rs1_value == rs2_value) begin
                branch_taken  = 1'b1;
```

```
                // avisa que vai saltar
                branch_target = pc_ex + branch_imm;
            end
        end
    end
```

Listing 1: Lógica combinacional corrigida na BranchUnit

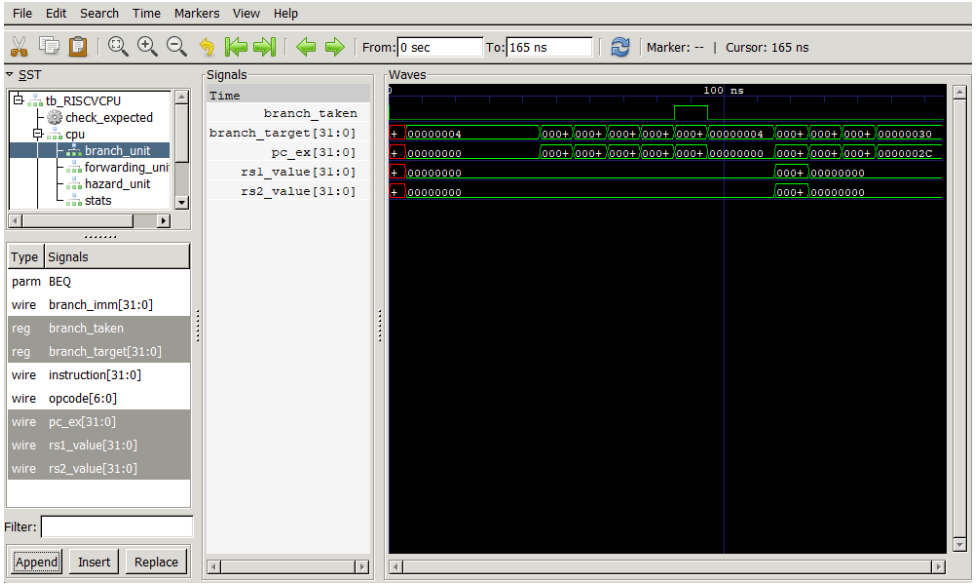

Figura 1: GTKWave demonstrando o sinal branch_taken e o cálculo do PC de destino corrigidos.

Após a modificação, a simulação confirmou o funcionamento do desvio, resultando na limpeza bem-sucedida de uma instrução indevida do pipeline (1 *flush*):

`Cycles : 14 | Instructions : 7 | Branches : 1 | Flushes : 1 | CPI : 2.00`

3 Parte 2: Resolução via Software (NOPs)

Durante a execução de instruções sequenciais dependentes, identificamos conflitos do tipo *Read After Write* (RAW). Como a arquitetura inicial não possuía uma unidade de *forwarding* para adiantar os resultados, foi necessário intervir via software, inserindo instruções "bolha"(`NOP`) manualmente no código Assembly.

3.1 Atraso de Escrita no Banco de Registradores

Durante a análise de temporização, identificou-se uma característica intrínseca do simulador *Icarus Verilog* utilizado: o sistema não realiza a separação clássica de leitura e escrita no estágio de *Write Back* (WB) dentro de metades opostas do mesmo ciclo de *clock*.

Devido a essa limitação síncrona (onde leituras e escritas ocorrem simultaneamente na borda de subida), o dado escrito no ciclo 5, por exemplo, só se torna estável e disponível para leitura no ciclo 6. Por este motivo, a implementação de apenas 2 NOPs demonstrou-se insuficiente, forçando a adoção de **3 NOPs** entre cada par de instruções dependentes para garantir a coerência dos dados lidos no estágio ID.

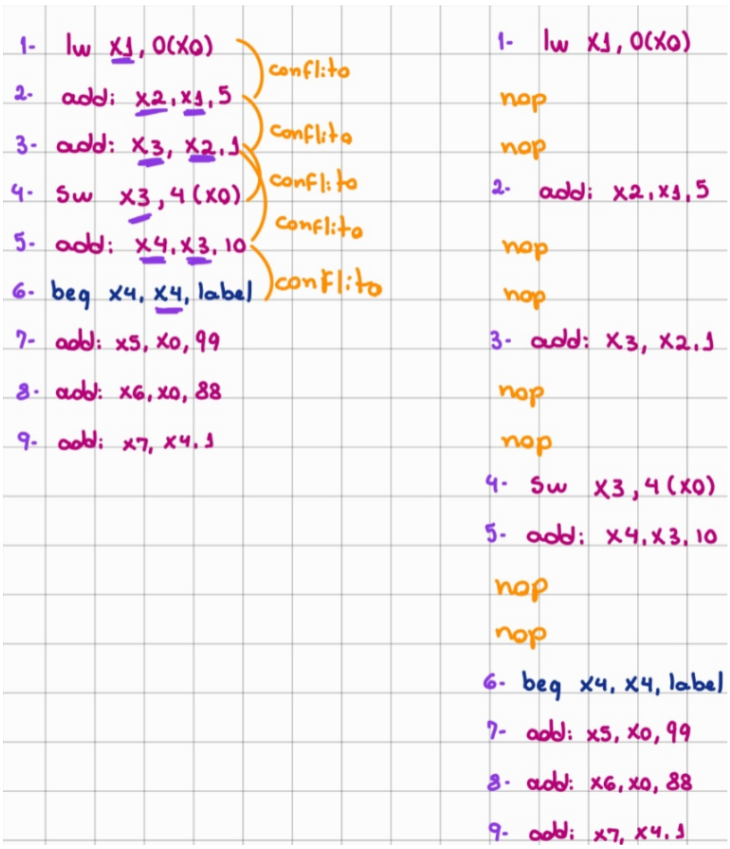

Figura 2: Mapeamento de bolhas manuais para sincronização de dados considerando a latência do simulador.

A mitigação via software resolveu as inconsistências matemáticas, porém degradou severamente a eficiência do pipeline:

```
--- LOG DE EXECUÇÃO COM NOPS ---
Cycles : 26 | Instructions: 7 | CPI: 3.71
PASS: all expected results match.
```

4 Parte 3: Resolução via Hardware

Para restaurar a eficiência do processador, o código Assembly foi retornado ao seu estado original (sem `NOPs`), transferindo a responsabilidade de resolução de conflitos para a própria arquitetura de hardware.

4.1 Forwarding Unit (Adiantamento)

Foi implementada a lógica de *Bypassing* na `ForwardingUnit`. A unidade monitora os registradores fonte no estágio de Execução (`idex_rs1` e `idex_rs2`). Caso esses registradores coincidam com os registradores de destino de instruções que estão nos estágios de Memória (`exmem_rd`) ou *Write Back* (`memwb_rd`) — garantindo que não seja o registrador nulo `x0` —, multiplexadores redirecionam os dados recém-calculados diretamente para a entrada da ALU.

```
    always @(*) begin
        forwardA = NO_FORWARD;
```

```
        forwardB = NO_FORWARD;
        // logica do forwarding para operando A
        if ((exmem_rd != 5'd0) && (idex_rs1 == exmem_rd)) begin
            forwardA = FROM_MEM;
        end
        else if ((memwb_rd != 5'd0) && (idex_rs1 == memwb_rd)) begin
            if (memwb_op == ALUop)
                forwardA = FROM_WB_ALU;
            else if (memwb_op == LW)
                forwardA = FROM_WB_LD;
        end

        // logica do forwarding para operando B
        if ((exmem_rd != 5'd0) && (idex_rs2 == exmem_rd)) begin
            forwardB = FROM_MEM;
        end
        else if ((memwb_rd != 5'd0) && (idex_rs2 == memwb_rd)) begin
            if (memwb_op == ALUop)
                forwardB = FROM_WB_ALU;
            else if (memwb_op == LW)
                forwardB = FROM_WB_LD;
        end
    end
```

Listing 2: Lógica de roteamento de dados e tratamento de conflitos duplos na ForwardingUnit

4.2 Hazard Detection Unit (Stalls)

Para o *Hazard* específico de carga (*Load-Use*), o adiantamento não é suficiente, visto que o dado provém da memória principal e possui maior latência. A `HazardDetectionUnit` foi desenvolvida para detectar se a instrução no estágio de Memória é um Load (`exmem_op == LW`) e se o seu destino coincide com as fontes da instrução atualmente em Execução. Nesse cenário isolado, o hardware injeta automaticamente uma bolha de atraso ($stall = 1$).

```
    always @(*) begin
        stall = 1'b0;
        // Load-use hazard:
        // If the instruction in EX/MEM is a load and the instruction in ID/EX
        // uses its destination register, the pipeline must stall one cycle.
        // logica para detectar hazard causado por load
        if (exmem_op == LW) begin
            if ((exmem_rd == idex_rs1) || (exmem_rd == idex_rs2)) begin
                stall = 1'b1;
            end
        end
    end
```

Listing 3: Lógica de injeção de bolhas para Load-Use Hazard na HazardDetectionUnit

5 Análise Crítica dos Resultados

A avaliação do desempenho do processador antes e após a implementação da resolução de *hazards* em hardware demonstra um ganho significativo de eficiência. Os dados extraídos do simulador estão consolidados na Tabela 1.

Tabela 1: Comparativo Geral de Métricas do Pipeline

Métrica	**Solução Software (NOPs)**	**Solução Hardware (Final)**
Ciclos Totais	26	**15**
Instruções Úteis	7	7
Bypasses Utilizados	0	**6**
Stalls Automáticos	0	**1**
CPI Final	**3.71**	**2.14**

A partir dessas métricas, destacamos as seguintes conclusões técnicas:

- **Cálculo de Speedup:** A substituição das bolhas de software (`NOPs`) pelo roteamento dinâmico de dados (*Forwarding*) reduziu a contagem de ciclos de 26 para 15. Isso representa um *Speedup* calculado pela fórmula $S = \frac{Tempo_{antigo}}{Tempo_{novo}} = \frac{26}{15} \approx 1,73\times$. Ou seja, o processador otimizado é **73% mais rápido** na execução da mesma carga de trabalho.

- **Otimização do CPI (Ciclos por Instrução):** Em um pipeline ideal, o limite teórico de desempenho é um CPI de 1,0. Na abordagem via software, o CPI foi de 3,71, evidenciando o severo desperdício de ciclos ociosos. Com a arquitetura inteligente, o CPI caiu drasticamente para 2,14. O valor final não atinge 1,0 perfeitamente devido ao *overhead* natural de preenchimento inicial do pipeline (*fill*) e ao *stall* obrigatório de 1 ciclo inserido pela `HazardDetectionUnit` para resolver a restrição física de memória (*Load-Use*).

- **Eficácia Cirúrgica do Hardware:** O registro exato de 6 *bypasses* confirma que a `ForwardingUnit` atuou de forma exaustiva para salvar ciclos. Por outro lado, o único atraso registrado (*stall* = 1) atesta que a lógica combinacional soube discernir com precisão a única situação onde o adiantamento elétrico era fisicamente impossível.

Conclusão: O *trade-off* entre o aumento da complexidade de área do hardware (adição de multiplexadores, fios de atalho e portas lógicas de controle) e o ganho de desempenho obtido provou-se altamente vantajoso. A arquitetura final demonstrou-se robusta, preservando a integridade dos resultados (todos os testes com *PASS*) enquanto maximizava a vazão das instruções.

A Dificuldades Encontradas

Durante o desenvolvimento, a equipe encontrou desafios na configuração inicial do ambiente Icarus Verilog no sistema operacional Windows. Além disso, o comportamento atípico do simulador em relação à temporização do *Write Back* (que não divide o ciclo em metades) exigiu um esforço extra de *debugging* para identificar a necessidade de injetar 3 NOPs ao invés dos 2 previstos na literatura teórica. Por fim, houve um desafio na compreensão da persistência de `NOPs` ocultos na memória ao transitar da Parte 2 para a Parte 3, que mascarava a ativação correta do Forwarding no log de estatísticas.

B Uso de IA e Modelo Spec-Driven Development (SDD)

O modelo de Inteligência Artificial Gemini (versão Pro 3.1) foi utilizado como ferramenta de tutoria seguindo as diretrizes exigidas na Seção 10.2. A IA auxiliou na análise estrutural do tempo do pipeline (visualização de ciclos), depuração de erros de sintaxe em Verilog, refinamento da lógica booleana dos arquivos de hardware e, por fim, na estruturação textual e formatação deste relatório em Latex. As interações não foram compostas por solicitações de geração de código genérico, mas sim por validações de hipóteses estruturais e especificações de limites físicos.

Nota sobre Transparência e Compartilhamento: O compartilhamento de link público do chat foi desabilitado pela plataforma devido ao carregamento de arquivos locais (PDFs e imagens de código) para análise de contexto. Devido à extensão das interações contínuas necessárias para manter o escopo técnico do processador, os prompts estruturais de maior impacto arquitetural foram transcritos e justificados abaixo. A transcrição integral de todas as interações menores e complementares encontra-se disponível no arquivo `historico_chat_ia.md` presente no nosso repositório no GitHub.

Repositório do Projeto no GitHub: `https://github.com/DebLuiza/TP02_ACIII/tree/main/trabalho_pipeline/prompts`

B.1 Registro de Prompts e Alinhamento ao SDD

Abaixo, transcrevemos as interações reais da equipe com a ferramenta, acompanhadas da justificativa de como se alinham à especificação técnica exigida:

- **Prompt (Escopo e Planejamento):** *"Estes são os arquivos oficiais do trabalho de arquitetura de computadores, quero que você seja meu auxiliar neste trabalho. Gostaria primeiramente que você me explicasse o objetivo geral e dividisse em tópicos os próximos passos que eu deveria realizar..."*
  Justificativa SDD: Estabelece o contexto e as **restrições de interface** ao fornecer os arquivos oficiais. Define o requisito de planejar o desenvolvimento em etapas lógicas antes de qualquer implementação.

- **Prompt (Análise de Requisitos e Interface):** *"ah sim, perfeito vamos começar com a primeira tarefa então, a instrução beq serve para desvio correto? como se fosse um if e else, como eu não estou muito acostumada com verilog, gostaria que me explicasse o que foi feito até o momento no arquivo BranchUnit.v"*
  Justificativa SDD: Define o **problema** (entender o Hazard de Controle) e impõe a **restrição** de linguagem (Verilog) e de localidade (arquivo `BranchUnit.v`).

- **Prompt (Definição de Problema e Timing):** *"Instrução 1 IF ID EX MEM WB | Instrução 2 0 IF ID EX MEM | Instrução 3 0 0 IF ID EX. Quando a instrução 2 pega o valor x1, a instrução 1 ainda não completou seus ciclos (WB)... seria esse o problema, correto?"*
  Justificativa SDD: Especifica tecnicamente a **restrição de timing** e o **problema funcional** de dependência RAW, validando a lógica do pipeline antes de propor a solução.

- **Prompt (Especificação de Requisito Funcional):** *"A função do Forwarding é pegar um dado que acabou de ser calculado mas que ainda não foi gravado no registrador e adiantar para a entrada da ALU, certo? mas como eu sei de qual estágio*

para qual estágio eu posso fazer isso?"
Justificativa SDD: Especifica claramente o **requisito funcional** desejado (adiantamento de dados) e solicita a definição das **restrições físicas** dos estágios do pipeline.

- **Prompt (Teste de Limites e Casos de Borda):** *"então se uma instrução 1 estiver no estágio 4 e a instrução 2 no estágio 3 podemos fazer forward? tipo: add x13, x11, x12 | lw x13 0(x0) ?"*
  Justificativa SDD: Cria um **caso de teste específico** para validar se a especificação de *forwarding* possui restrições para instruções de escrita em memória (`LW`), evitando implementações incorretas.

- **Prompt (Validação de Lógica e Sintaxe):** *"if (idex_op == LW && (exmem_rd == idex_rs1 || exmem_rd == idex_rs2)) begin stall = 1'b1; end dessa forma?"*
  Justificativa SDD: Submissão de proposta técnica para validação. Demonstra que o grupo detém a **responsabilidade técnica** (Seção 10.3) ao propor a lógica booleana e usar a IA apenas para conferência de sintaxe e sinais.

- **Prompt (Validação de Interface):** *"module HazardDetectionUnit (...) porém ifid_rs2 não está no código"*
  Alinhamento SDD: Demonstra restrição de implementação e interface. O grupo confrontou a teoria clássica com as interfaces reais disponíveis no arquivo HazardDetectionUnit.v fornecido no TP, ajustando a lógica combinacional para `exmem_op`.

- **Prompt (Depuração de Restrição de Timing):** *"[Envio de log com estatísticas] vou rodar os testes primeiro... o que acha? (...) aqui estão os testes comentando os nops e com as outras modificações... vou te enviar meus arquivos modificados, qual pode ser o erro?"*
  Alinhamento SDD: O envio dos logs com 0 Bypasses serviu para investigar falhas de *timing*. Isso resultou na descoberta de que a função `clear_memories()` do *testbench* estava mascarando a latência preenchendo as quebras com NOPs na memória de instruções.